

**Robot Sumo Simulation:
Milestone 1 Document**

Team Members

Adebare Kofo-Abayomi: akofoaabyomi2022@my.fit.edu

Avlynn Wade: awade2024@my.fit.edu

Faysal Zaki: fzaki2023@my.fit.edu

Joshua Oziel Fernández Alvarado: jfernandezal2023@my.fit.edu

Client & Faculty Advisor

Department of Electrical Engineering and Computer Science

Dr. Hanks, thanks@fit.edu

Section 1.0 Progress of Current Milestone

Figure 1. Progress Matrix

Task	Completion %	Alex	Avlynn	Faysal	Joshua	To do
Create demos with all engines	100%	N/A	N/A	50%	50%	None
Research RL API Integration	100%	50%	50%	N/A	N/A	None
Requirement Document	100%	20%	40%	20%	20%	None
Design Document	100%	20%	20%	20%	40%	None
Test Document	100%	25%	25%	25%	25%	None

Section 2.0 Discussion of Milestone 1 Tasks

Section 2.1 Create Engine Demos

A demo for each of the three candidate engines (Unity, Godot, and Pygame) was developed in order to evaluate the compatibility of each one for the project going forward. Demos consist of a simple short match between two robots where one robot pushes the other, and a winner for the match is determined. The Unity demo brought some difficulty because of the developers' unfamiliarity with it. The Pygame demo brought some difficulty because of the lack of easy physics implementation. In the end, the developers chose to continue the project using the Godot Engine for its integrated physics engine and useful classes.

Section 2.2 Research RL API Integration

The four selected RL APIs from the Project Plan: Gymnasium, PettingZoo, MOMAland, and VMAS (Vectorized Multi-Agent Simulator), were researched in order to evaluate their application to the proposed project. After said evaluation, the development team selected Gymnasium as the API for the single-agent baseline due to its simplicity, ease of use, and overall suitability for the planned single-agent approach. Similarly, PettingZoo was selected as the multi-agent environment due to its overall compatibility with the proposed MARL functionality, as well as its ease of use and ample documentation. The VMAS API would have been more difficult to implement as it is constrained to 2D applications and is more of a physics engine rather than a dedicated RL API. The MOMAland API had similar capabilities to PettingZoo and could have been chosen for the multi-agent option; but ultimately PettingZoo was selected due to its similarity with the chosen Gymnasium API and ease of integration (as highlighted above).

Section 2.3 Create Requirement Document

The requirement document, and all of its corresponding sections, was first outlined using the suggested “IEEE Guide for Software Requirements Specifications”. From here, the sections were drafted, taking into consideration the numerous interfaces, functional requirements, and performance requirements of the Robot Sumo project. Finally, annotations from the development team were proposed, and a final draft was created and submitted to the faculty advisor for review and approval. One challenge encountered was making the requirements documentation consistent with the language from both the design and test documents. However, this challenge was mitigated by ample discussion amongst the development team.

Section 2.4 Create Design Document

The Design document was first outlined with the most important sections pertaining to the project. Sections were written after drafting a general overview of the system architecture, and corresponding diagrams were drawn and added afterwards. All sections were afterwards revised as a group. Specifics of how the system will look were decided by the development team through verbal and written discussion.

Section 2.5 Create Test Document

The test document was drafted asynchronously with the requirements document. By the time the first draft of the test document was completed, it was outdated and its contents didn’t match what was outlined in the requirements document. A 2nd draft of the test document was made and formatted to better meet the IEEE standard.

Section 3.0 Discussion of Team Contributions

Section 3.1 Adebare Kofo-Abayomi

Responsible for completing research on the PettingZoo and MOMAland RL APIs evaluation. He compared their multi-agent features and suitability for the project, contributing to the team’s selection of PettingZoo. Adebare also helped establish the initial structure of the presentations that was later edited by the team. He was also responsible for part of the test document, including the requirement-linked prototype cases and planned coverage for later milestones.

Section 3.2 Avlynn Wade

Responsible for completing the RL API research as well as writing the requirements document. Specifically, Avlynn was in charge of researching the Gymnasium and Vectorized Multi-Agent Simulator (VMAS) RL API’s and filling in the corresponding documentation. Additionally, Avlynn sectioned out, drafted, and then finalized the requirements document, taking into account input from the development team. Avlynn also helped in planning the next

Milestone (2), as well as filling out the Milestone 1 document and the corresponding presentation.

Section 3.3 Faysal Zaki

Faysal helped edit and correct the finished rough draft of the requirements document. The edits trimmed the draft down to be more concise and accurate. There were also minor edits that corrected technical definitions. Faysal also wrote the 2nd and final draft of the test document. The 2nd draft was a total rewrite of the 1st draft. The 2nd draft was aimed at improving the formatting and conforming to the IEEE standard.

Section 3.4 Joshua Oziel Fernández Alvarado

Joshua developed the Godot Engine and Pygame demo simulations, pushing all commits of their development to the shared GitHub repository. This included providing the feedback about the engines and exploring alternative design options utilizing the obtained experience. Joshua sectioned, drafted, and finalized the design document with design decisions discussed among the team members. Finally, Joshua contributed to writing the Milestone 1 document, planning for Milestone 2, and making the presentation.

Section 4.0 Task Matrix for Milestone 2

Task	Avlynn	Joshua	Faysal	Adebare
Implement robot models with corresponding hitboxes.	10%	10%	70%	10%
Give robots the ability to interact with the environment and set start and end conditions for a sumo match.	10%	70%	10%	10%

Give exposable inputs for the robot controller.	10%	70%	10%	10%
Provide readable full state feedback for RL training.	10%	10%	10%	70%
Implement a basic interface for the simulation and RL API.	70%	10%	10%	10%
Research specific RL algorithms that are best suited to the project.	70%	10%	10%	10%

Section 5.0 Discussion for Planned Milestone 2 Tasks

Section 5.1 Implement robot models with corresponding hitboxes

A framework that allows for robot models to be imported and applied to a robot instance, instantiating corresponding hitboxes which correctly represent the robot. Although modular robots are currently out of scope, build a scalable system that will allow for multiple robot parts to work together and for the hitboxes to correctly conjoin. Allow for a quick export of the robot dimensions so it can be part of the RL input later.

Section 5.2 Give robots the ability to interact with the environment and set start and end conditions for a sumo match

Define all the possible ways a sumo match can end and implement them as conditions independent of the match settings, so any number of robots, robot types, and match environment can be used while still correctly making decisions regarding wins and losses. Allow robots the

ability to interact with potential obstacles and other environmental changes in the ring for more complex training.

Section 5.3 Give exposable inputs for the robot controller

Define a controller for the robot that allows input to be received by the agent or a user. Input should be interpreted and applied with physics, not a direct change of the robot's transform, just like how a real remote controller would work. The controller should have different modes of input to account for robot types, so the player and agent can both control a robot.

Section 5.4 Provide readable full state feedback for RL training

Define how a match's current state can best be represented using as little information as possible and transform it into readable information that the RL API can interpret. Ensure the feedback is easily readable and can be integrated into the training loop. Define how the RL API will receive feedback.

Section 5.5 Implement a basic interface for the simulation and RL API

Implement an interface that acts as a bridge between the RL API and the Godot simulation. It should also bridge the language gap in Godot through the use of a Godot to Python extension or other means necessary. It should correctly execute a training loop through, including the input and feedback steps.

Section 5.6 Research specific RL algorithms that are best suited to the project

The development team will need to research the specific RL algorithms needed to bridge the gap between the outlined Gymnasium API and the learned Robot Sumo behavior. The development team will look into the best-practice approaches for single-agent learning, as well as investigate any useful RL libraries (Stable-Baselines3 (SB3), PyTorch, RLLib). Additionally, the development team will explore how these specific algorithms are integrated into the selected API. The results of this research will be outlined and presented in a clear format for team evaluation.

Section 6.0 Dates of Meetings with Faculty Advisor (and Client) During Milestone 1

Meeting Purpose	Date and Time
Progress Review and Engine Evaluation	September 11th, 2026 – 12:00 pm
Approval of Milestone 1 Documents	September 27th, 2026 – conducted virtually

Note: Throughout Milestone 1, the faculty advisor (and client) was in constant communication with the development team through the following communication tools: discord, email, and shared documents (Google Drive-based sharing).

Section 7.0 Client and Faculty Advisor Feedback for Milestone 1

Note: the Client and Faculty Advisor are the same for this proposed project.

Section 7.1 Feedback: Create Engine Demos

The engine demos look good and help demonstrate the pros and cons of each choice. It would have been nice to see the unity demo developed to the same level as the godot and pygame demos (e.g. including win detection and different colors for robots), but it at least proves the concept.

Section 7.2 Feedback: Research RL API Integration

I approve of the choice of Gymnasium and PettingZoo as the RL API choices. I think these are the best supported and canonical choices, so they should offer the least friction and easiest integration of different RL algorithms. I appreciate the consideration of alternative choices.

Section 7.3 Feedback: Create Requirement Document

The requirement document is very well organized and detailed. It considers the many functional and non-functional requirements of the project and designates important user personas. I especially appreciate that it considers not only general feature requirements, but also requirements involving training pipelines on AI.Panther. The faculty advisor will be requesting student access to the cluster to facilitate delivery of these requirements.

Section 7.4 Feedback: Create Design Document

The overall system architecture looks sound as supported by the class diagrams and mock GUI. A section on data management was included, which is very important for experimental RL projects. Going forward, I think the highest-risk technical integration point will be the interface between the Godot engine and the Python RL training environment, so it will be nice to see how this gets fleshed out going forward.

Section 7.5 Feedback: Create Test Document

The test document overall looks good. It mostly focuses on RL training, physics, and GUI/configuration functionality. As the project progresses, I would like to see some testing plans

around data/failure handling, HPC submission/retrieval, reproducibility, and the various non-functional requirements mentioned in the requirements doc. It would be nice to eventually have a requirement-to-test traceability matrix so that we can track which tests exercise which requirements.